

Especificación Técnica y Ecosistema de Data Products

Arquitectura Medallion y Data Mesh en la Gestión de Inventarios por Consignación

Maestría en Ingeniería de Datos
Universidad de Ingeniería y Tecnología - UTEC Posgrado
Grupo G2 (Sección 1 - g102)

18 de Julio de 2026

Índice

1. Introducción y Contexto de Negocio	2
2. El Ecosistema de Data Products (Data Mesh)	3
2.1. Conexiones y Dependencias Semánticas	3
2.2. Diagrama del Ecosistema de Datos	3
3. Detalle de Procesamiento por Notebook	4
3.1. Notebook 1: Ingesta Inmutable (Capa Bronze)	4
3.1.1. Diagrama de Flujo (Bronze)	4
3.1.2. Pseudocódigo (Bronze)	4
3.2. Notebook 2: Calidad, Enmascaramiento y Cuarentena (Capa Silver)	6
3.2.1. Diagrama de Flujo (Silver)	6
3.2.2. Pseudocódigo (Silver)	6
3.3. Notebook 3: Creación de Vistas (Capa Gold)	8
3.3.1. Diagrama de Flujo (Gold)	8
3.3.2. Pseudocódigo (Gold - SQL)	8
3.4. Notebook 4: Gobernanza, SLAs y Data Contracts	10
3.4.1. Diagrama de Flujo (Gobernanza)	10
3.4.2. Pseudocódigo (Gobernanza)	10

1. Introducción y Contexto de Negocio

El presente documento describe detalladamente el diseño, la lógica de procesamiento y la arquitectura del pipeline de datos implementado para la **Compañía Minera Los Andes S.A.**. El objetivo del sistema es automatizar y asegurar el abastecimiento de repuestos críticos bajo el modelo logístico de **Consignación** en mina (donde el inventario pertenece al proveedor hasta que la minera lo consume en el tajo abierto o planta concentradora).

Para resolver los riesgos de desabastecimiento, optimizar el capital de trabajo y agilizar la facturación mensual, se implementaron **3 Data Products** en una arquitectura Medallion descentralizada (**Data Mesh**) utilizando Databricks y Unity Catalog.

2. El Ecosistema de Data Products (Data Mesh)

En un modelo de Data Mesh, los productos de datos no operan de forma aislada. Comparten una capa unificada y se retroalimentan mutuamente para refinar las políticas operativas y contables de la empresa.

2.1. Conexiones y Dependencias Semánticas

- **Alimentación de Origen:** Los tres Data Products en la capa Gold se alimentan de la tabla unificada enriquecida de la capa Silver (`silver_movimientos_consignacion`).
- **Ajuste Dinámico de Alertas (DP2 → DP1):** El DP2 (**Análisis de Consumo**) calcula el desgaste mensual real de los materiales. Si se detecta un incremento sostenido en el consumo de una categoría (ej. medios de molienda), Planeamiento de Mantenimiento ajusta al alza el Stock de Seguridad y el Punto de Reorden en el catálogo maestro de materiales. Como resultado, las alertas del DP1 (**Reabastecimiento**) se adaptan dinámicamente al ritmo de consumo del tajo.
- **Auditoría de Reconciliación (DP2 ↔ DP3):** El costo financiero mensual de los retiros reportados en el DP2 debe conciliarse centavo a centavo con los documentos de material listos para facturar del DP3 (**Pagos**). Finanzas audita que no existan discrepancias entre ambos productos antes de emitir la orden de pago.

2.2. Diagrama del Ecosistema de Datos

A continuación se ilustra la topología del Data Mesh implementada:

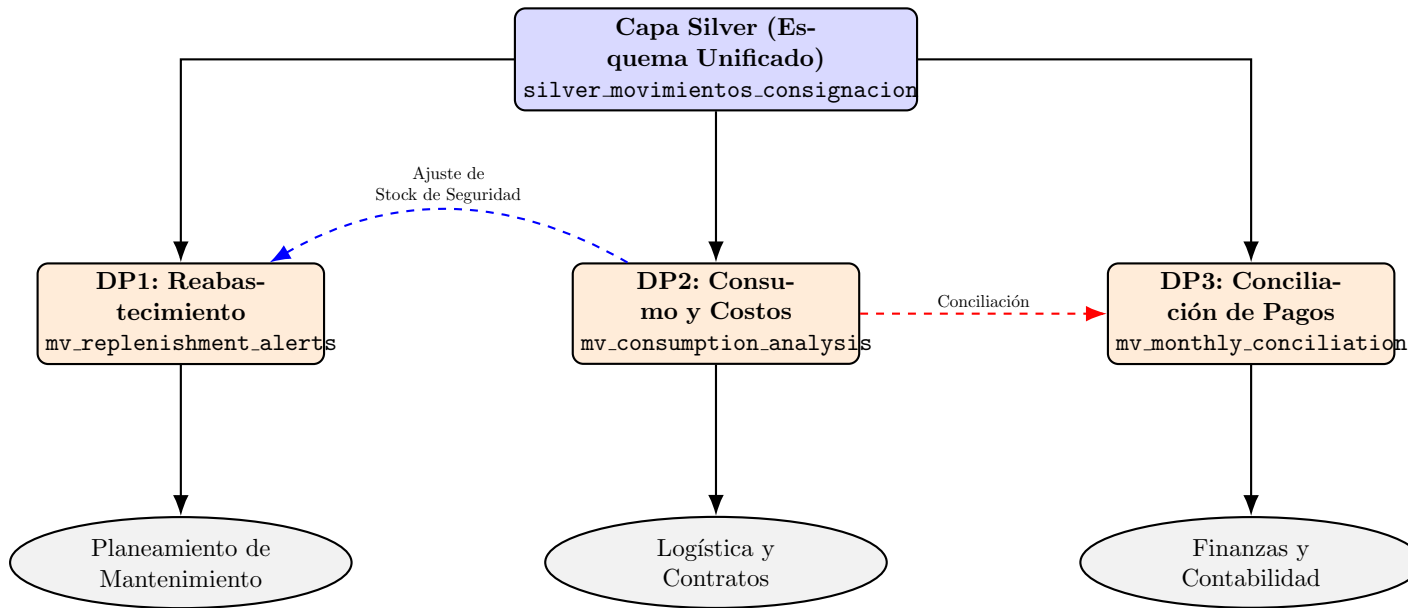


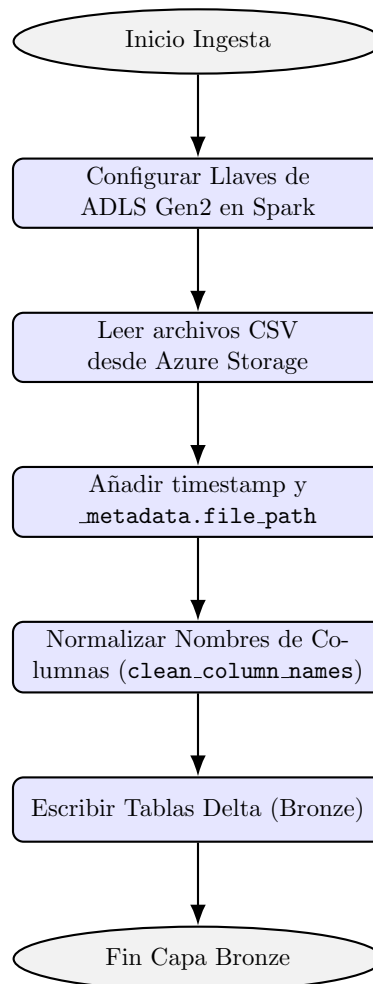
Figura 1: Mapa conceptual de Data Mesh y dependencias entre Data Products.

3. Detalle de Procesamiento por Notebook

3.1. Notebook 1: Ingesta Inmutable (Capa Bronze)

Este notebook realiza la ingesta inmutable de los archivos transaccionales de SAP y el catálogo maestro de materiales hacia tablas Delta gobernadas en Unity Catalog.

3.1.1. Diagrama de Flujo (Bronze)



3.1.2. Pseudocódigo (Bronze)

```
1 # Configuración inicial de acceso
2 spark.conf.set(
3     "fs.azure.account.key.stdemdsai.dfs.core.windows.net",
4     "STORAGE_KEY"
5 )
6
7 # Lectura de datos crudos
8 df_movimientos_raw = spark.read.format("csv") \
9     .option("header", "true") \
10    .option("inferSchema", "true") \
11    .load("abfss://.../raw/airflow/G2/MB51_*.csv")
12
13 df_maestro_raw = spark.read.format("csv") \
14    .option("header", "true") \
```

```

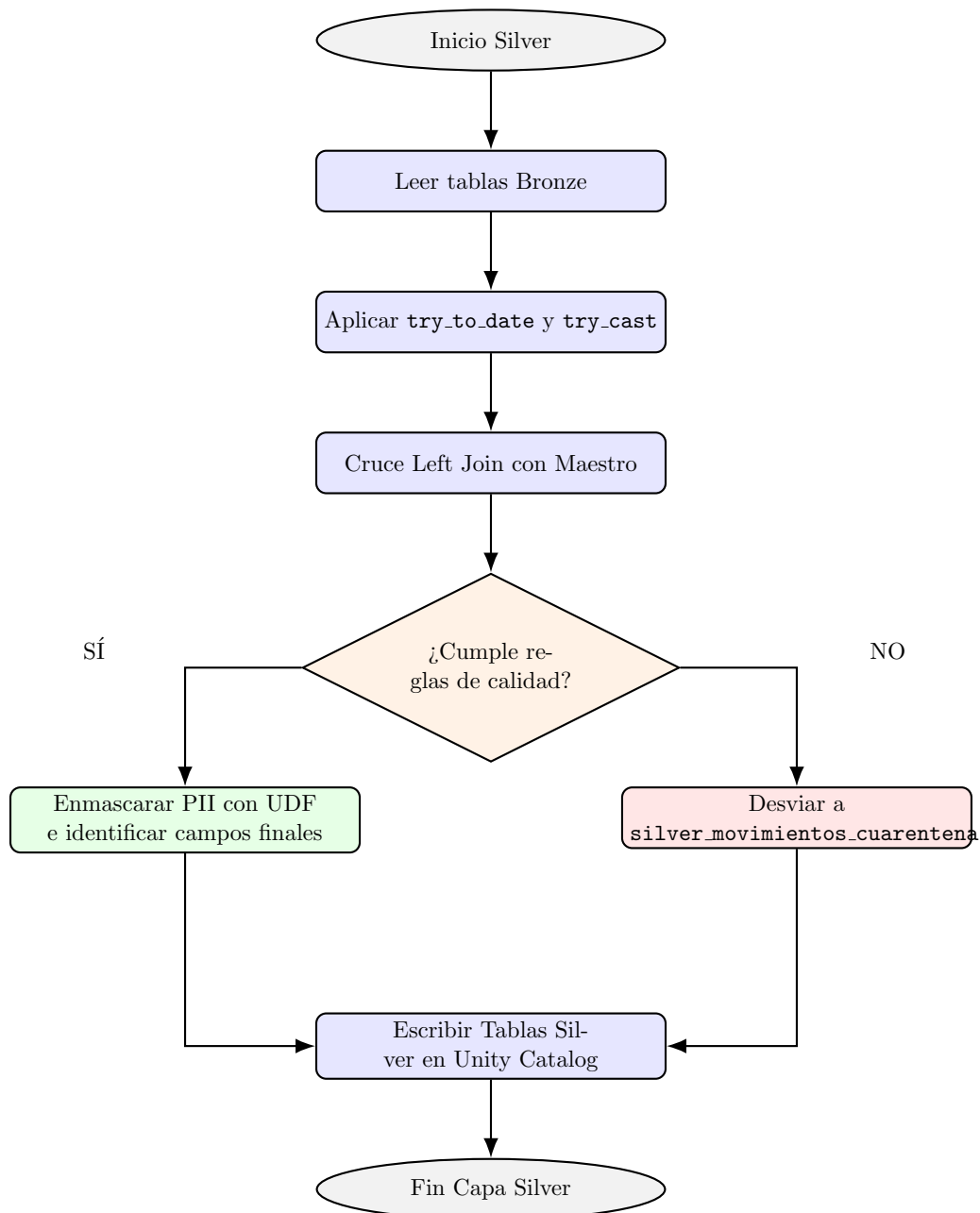
15     .option("inferSchema", "true") \
16     .load("abfss://.../raw/airflow/G2/maestro_materiales.csv")
17
18 # Normalizaci n de nombres de columnas
19 def clean_column_names(df):
20     for col_name in df.columns:
21         clean_name = col_name.strip() \
22             .replace(" ", "_") \
23             .replace(".", "_") \
24             .replace(";", "_") \
25             .replace("(", "") \
26             .replace(")", "")
27         df = df.withColumnRenamed(col_name, clean_name)
28     return df
29
30 df_movimientos_clean = clean_column_names(df_movimientos_raw)
31 df_movimientos_bronze = df_movimientos_clean \
32     .withColumn("ingested_at", current_timestamp()) \
33     .withColumn("file_source_name", col("_metadata.file_path"))
34
35 # Escritura inmutable
36 df_movimientos_bronze.write.format("delta") \
37     .mode("overwrite") \
38     .saveAsTable("g102_log_reabastecimiento.bronze.bronze_movimientos_sap")

```

3.2. Notebook 2: Calidad, Enmascaramiento y Cuarentena (Capa Silver)

Este notebook implementa las reglas de negocio de calidad de datos, detecta registros corruptos desviándolos a la tabla de cuarentena, y aplica enmascaramiento dinámico a los nombres de usuario (datos PII).

3.2.1. Diagrama de Flujo (Silver)



3.2.2. Pseudocódigo (Silver)

```
1 # Cargar datos de Bronze
2 df_mov = spark.table("bronze.bronze_movimientos_sap")
3 df_maestro = spark.table("bronze.bronze_maestro_materiales") \
4     .drop("ingested_at", "file_source_name")
5
```

```

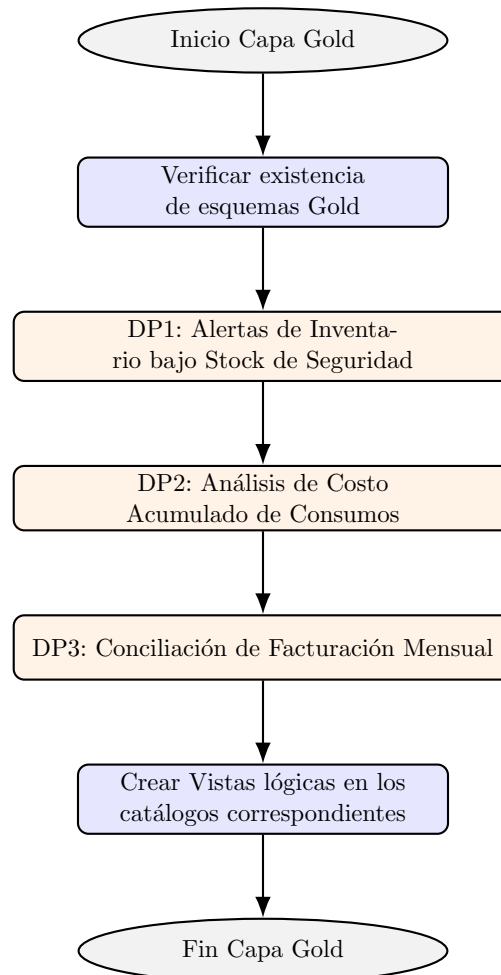
6 # Formateo y limpieza resiliente (try_cast y try_to_date)
7 df_mov_clean = df_mov.withColumn("Posting_Date", coalesce(
8     expr("try_to_date(Posting_Date, 'yyyy-MM-dd')"),
9     expr("try_to_date(Posting_Date, 'd/M/yyyy')")
10 ))).withColumn("Qty", expr("try_cast(Quantity AS DOUBLE)")) \
11     .withColumn("Amount_Loc_Cur", expr("try_cast(Amt_in_Loc_Cur_ AS DOUBLE)")) \
12     .filter(col("Special_Stock") == "K")
13
14 # Cruce con el maestro de materiales
15 df_joined = df_mov_clean.join(df_maestro, df_mov_clean.Material == df_maestro.
16     Material, "left")
17
18 # Reglas de desv o a Cuarentena
19 condicion_cuarentena = (col("Qty").isNull()) | (col("Qty") == 0) | \
20     (col("Proveedor").isNull()) | (col("Posting_Date").isNull
21     ())
22
23 df_cuarentena = df_joined.filter(condicion_cuarentena)
24 df_silver_clean = df_joined.filter(~condicion_cuarentena)
25
26 # Escribir en Cuarentena
27 df_cuarentena.select("Material_Id", "Posting_Date", "Qty", "Storage_Location") \
28     .write.mode("append").saveAsTable("silver.silver_movimientos_cuarentena")
29
30 # Enmascaramiento dinámico PII y selección final
31 df_silver_final = df_silver_clean.withColumn("usuario_sap_masked", expr("silver.
32     pii_mask_user(User_Name)"))
33 df_silver_final.select("material_id", "descripcion", "proveedor", "cantidad", "
34     costo_unitario_usd", "usuario_sap_masked") \
35     .write.mode("overwrite").saveAsTable("silver.silver_movimientos_consignacion
36     ")

```

3.3. Notebook 3: Creación de Vistas (Capa Gold)

Este notebook estructura los datos refinados de la capa Silver en vistas semánticas orientadas al negocio que definen a los 3 Data Products.

3.3.1. Diagrama de Flujo (Gold)



3.3.2. Pseudocódigo (Gold - SQL)

```
1 -- 1. DATA PRODUCT 1: ALERTAS DE REABASTECIMIENTO CR TICO
2 CREATE OR REPLACE VIEW g102_log_reabastecimiento.gold.mv_replenishment_alerts AS
3 WITH stock_acumulado AS (
4     SELECT material_id, descripcion, proveedor, categoria, stock_seguridad,
5           punto_reorden, costo_unitario_usd, SUM(cantidad) AS stock_actual
6     FROM g102_log_reabastecimiento.silver.silver_movimientos_consignacion
7     GROUP BY material_id, descripcion, proveedor, categoria, stock_seguridad,
8           punto_reorden, costo_unitario_usd
9 )
10 SELECT material_id, descripcion, proveedor, categoria, stock_actual,
11        stock_seguridad, punto_reorden,
12        CASE
13            WHEN stock_actual < stock_seguridad THEN 'CRITICO - DESABASTECIMIENTO'
14            WHEN stock_actual < punto_reorden THEN 'REORDEN SUGERIDO'
15            ELSE 'NORMAL'
16        END AS estado_inventario,
17        (punto_reorden - stock_actual) AS cantidad_a_comprar,
```



```

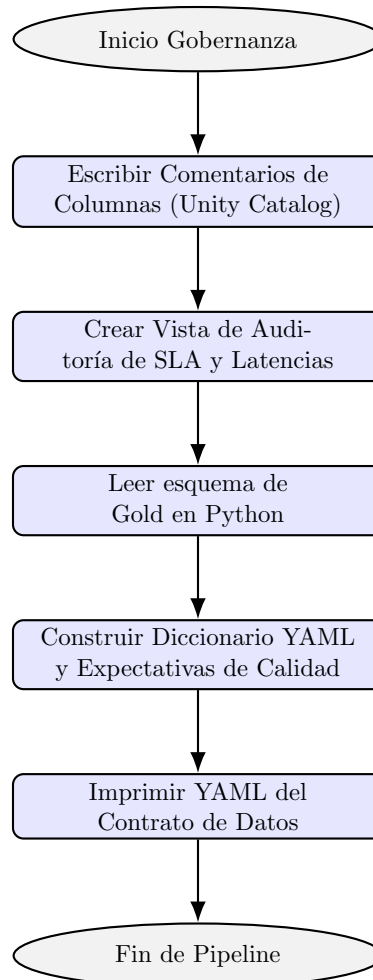
17 ((punto_reorden - stock_actual) * costo_unitario_usd) AS costo_reposicion_usd
18 FROM stock_acumulado
19 WHERE stock_actual < punto_reorden;
20
21 -- 2. DATA PRODUCT 2: CONSUMO MENSUAL Y COSTOS
22 CREATE OR REPLACE VIEW g102_log_consumos.gold.mv_consumption_analysis AS
23 SELECT date_format(fecha_movimiento, 'yyyy-MM') AS anio_mes, material_id,
24         descripcion, proveedor, categoria,
25         ABS(SUM(cantidad)) AS cantidad_consumida,
26         ABS(SUM(monto_total_usd)) AS costo_consumo_usd
27 FROM g102_log_reabastecimiento.silver.silver_movimientos_consignacion
28 WHERE cantidad < 0
29 GROUP BY date_format(fecha_movimiento, 'yyyy-MM'), material_id, descripcion,
30         proveedor, categoria;
31
32 -- 3. DATA PRODUCT 3: CONCILIACION MENSUAL DE PAGOS
33 CREATE OR REPLACE VIEW g102_fin_conciliacion.gold.mv_monthly_conciliation AS
34 SELECT date_format(fecha_movimiento, 'yyyy-MM') AS anio_mes, proveedor,
35         documento_material, material_id, descripcion, costo_unitario_usd,
36         ABS(SUM(cantidad)) AS cantidad_facturable,
37         ABS(SUM(monto_total_usd)) AS monto_facturable_usd
38 FROM g102_log_reabastecimiento.silver.silver_movimientos_consignacion
39 WHERE cantidad < 0
40 GROUP BY date_format(fecha_movimiento, 'yyyy-MM'), proveedor, documento_material
41         ,
42         material_id, descripcion, costo_unitario_usd;

```

3.4. Notebook 4: Gobernanza, SLAs y Data Contracts

Este notebook documenta los metadatos en Unity Catalog, estructura el reporte de monitoreo de SLAs de desempeño y compila el archivo YAML de contrato de datos para el Data Product.

3.4.1. Diagrama de Flujo (Gobernanza)



3.4.2. Pseudocódigo (Gobernanza)

```
1 # 1. Comentarios de Columnas en SQL
2 spark.sql("COMMENT ON COLUMN gold.mv_replenishment_alerts.material_id IS 'Codigo
   de material de SAP'")
3 spark.sql("COMMENT ON COLUMN gold.mv_replenishment_alerts.stock_actual IS '
   Inventario actual neto acumulado'")
4
5 # 2. Vista de Auditoría (Mockeada por permisos de system.query.history)
6 spark.sql("""
7 CREATE OR REPLACE VIEW gold.vw_dataproducts_audit AS
8 SELECT
9     current_date() AS start_date,
10     'Reabastecimiento Critico' AS data_product,
11     12 AS total_consultas,
12     1.45 AS avg_duracion_total_sec,
13     12 AS consultas_exitosas,
14     0 AS consultas_fallidas
15 UNION ALL
```

```

16 SELECT
17     current_date() AS start_date,
18     'Consumo Historico' AS data_product,
19     8 AS total_consultas,
20     2.10 AS avg_duracion_total_sec,
21     7 AS consultas_exitosas,
22     1 AS consultas_fallidas
23     "")
24
25 # 3. Data Contract en YAML
26 df_gold = spark.table("gold.mv_replenishment_alerts")
27 contract = {
28     "data_product": "reabastecimiento",
29     "domain": "logistica",
30     "schema": [],
31     "expectations": {
32         "freshness": "updated_daily",
33         "quality": [{"rule": "material_id IS NOT NULL", "level": "error"}]
34     }
35 }
36 for field in df_gold.schema.fields:
37     contract["schema"].append({
38         "name": field.name,
39         "type": field.dataType.simpleString(),
40         "description": field.metadata.get("comment", "")
41     })
42 print(yaml.dump(contract))

```